# TightWallet – Documento aggiornato (idee emerse + piano + KPI + bozza DB)

> Questo documento integra tutte le idee emerse oggi tra Matteo e Juan Pablo, con un taglio **analitico** (facile da leggere) ma **tecnico quanto basta** per trasformarlo in backlog.

---

## 0) Obiettivo in una frase

Costruire una app "alla GnuCash" (conti + partita doppia) ma con UX moderna, import iniziale dai tool già usati (**GnuCash** e **Money Manager**), dashboard componibile e processo "automatico ma supervisionato" per i pagamenti ricorrenti (anche variabili).

---

## 1) Visione e principi

**Principi non negoziabili**

- **Partita doppia** come base: ogni transazione è bilanciata (Dare = Avere).
- **Conti (Accounts)** come in GnuCash: Asset / Liability / Equity / Income / Expense.
- **Web-first**: su desktop va benissimo il browser; il mobile (app) chiama gli stessi endpoint.
- **Import subito, export dopo**: prima importiamo la storia (per partire con dashboard), poi pensiamo alle esportazioni.
- **Semplice per utenti "base", completo per utenti "avanzati"**.

**UX modes**

- **Classic** (Base): flussi semplici, niente jargon contabile.
- **Advanced**: mostra splits, ledger, report contabili.
- La modalità scelta va **salvata** nelle preferenze utente e ripristinata all'avvio.

---

## 2) Fondamenta contabili: Chart of Accounts (Assets / Liabilities / Equity / Income / Expenses)

### 2.1 Perché serve una struttura "standard"

- Evita conti/categorie create a caso.
- Rende import e KPI più robusti.
- Permette a chi vuole di ragionare in contabilità vera.

### 2.2 Template iniziale (consigliato)

Questa è una "base tipica" (personalizzabile):

**Assets (Attività)**

- Cash on Hand (Contanti)
- Bank Accounts

- Accounts Receivable (crediti)
- Other Current Assets
- Fixed Assets (beni durevoli)
- Investments / Mutual Funds / Stock

**Liabilities (Passività)**

- Accounts Payable (debiti)
- Credit Card (saldo carta come debito)
- Loans Payable (prestiti/finanziamenti)
- Other Liabilities

**Equity (Patrimonio netto)**

- Opening Balances (bilanci di apertura) ✅ fondamentale per inizializzare i conti
- Retained Earnings
- Owner Contributions / Owner Draws (se serve)
- Equity (generico)

**Income (Entrate)**

- Salary / Bonus
- Sales / Side income
- Interest / Dividends
- Other Income

**Expenses (Spese)**

- Utilities (luce/gas/acqua)
- Rent / Mortgage
- Groceries
- Transport (Telepass, carburante, parcheggi)
- Phone / Internet
- Subscriptions
- Taxes
- Health
- Leisure
- Other Expenses

### 2.3 Inizializzazione conti (Opening Balances)

Quando l'utente crea un nuovo conto (es. banca con 1.000€):

- si registra una transazione "Apertura":
  - Dare: `Assets:Bank` 1.000
  - Avere: `Equity:Opening Balances` 1.000

Così il saldo "nasce" correttamente senza scorciatoie.

---

## 3) Inserimento dati "stile GnuCash", ma user-friendly

### 3.1 Classic (Base)

Schermata “Aggiungi”:

- Tipo: **Spesa / Entrata / Trasferimento**
- Importo
- Conto “da” (o conto principale)
- Categoria (che in realtà è un conto Expense/Income)
- Data + descrizione facoltative

Esempi (dietro le quinte):

- Spesa Telepass:
  - Dare: `Expenses:Transport:Telepass`
  - Avere: `Assets:Card`
- Entrata stipendio:
  - Dare: `Assets:Bank`
  - Avere: `Income:Salary`

### 3.2 Advanced

- Permette transazioni con più righe (splits multipli).
- Permette vedere/modificare Dare/Avere.
- Accesso a:
  - Libro giornale (journal)
  - Libro mastro per conto (ledger)
  - Report contabili (Balance Sheet / Profit&Loss)

---

## 4) Import: partire dalla storia (GnuCash + Money Manager)

### 4.1 Obiettivo

Importare i dati già usati per:

- non ricominciare da zero
- avere subito dashboard e trend storici

### 4.2 Pipeline import (robusta e “sicura”)

1. Upload file / selezione sorgente
2. Parsing in area “staging”
3. Mapping (colonne → campi)
4. Normalizzazione (date, importi, valute, descrizioni)
5. Preview (cosa verrà creato + warning)
6. Commit atomico (crea conti/transazioni/splits)
7. Log errori e duplicati

### 4.3 Normalizzazione descrizioni (problema “Gas/gas/metano”)

Serve una logica per evitare dispersione:

- **Payee/Merchant canonico** (es. “GAS”) + alias (“Gas”, “metano”, “EstraGas”)
- Suggerimenti automatici mentre scrivi (autocomplete)
- Possibilità di “fissare”/“preferire” una descrizione (tipo “like”) e riusarla facilmente

Risultato: trend storici per “Gas” vengono fuori senza dover “estrarre a mano”.

---

## 5) Il dolore vero: “ogni mese devo ricordarmi cosa pago”

Qui entra la feature chiave.

### 5.1 Scheduled + Inbox (automatico supervisionato)

- Definisci una **Regola programmata** (Telepass, bolletta, rata).
- Il sistema genera una **Istanza attesa** nel mese (Inbox).
- Tu **approvi** / **modifichi** / **rimandi**.
- Solo dopo approvazione diventa transazione reale.

### 5.2 Pagamenti variabili: strategie importo

Per ogni regola scegli una strategia:

- **Fixed**: sempre uguale (Netflix).
- **LastMonth**: propone ultimo importo (Telepass/bollette).
- **AverageN**: propone media ultimi N mesi.
- **Estimate + Reconcile**: propone stima; quando arriva importo reale:
  - aggiorni l’importo **oppure**
  - registri differenza su un conto “Aggiustamenti”
- **Schedule Table**: tabella rate importabile (utile per finanziamenti tipo Findomestic con rate variabili/strutturate).

### 5.3 Match automatico con import estratti conto (in futuro)

Quando importi movimenti da banca/carta:

- match con “istanze attese” usando:
  - descrizione/merchant simile
  - importo vicino (tolleranza)
  - finestra date
- se match: istanza “matched” e zero lavoro manuale (solo controllo)

---

## 6) Dashboard componibile (come la vuole l’utente)

### 6.1 Regola

Dashboard = blocchi (card/widget) configurabili e riordinabili.

### 6.2 Widget base (Classic)

- Saldo totale
- Saldo per conto principale
- Entrate mese / Spese mese / Risparmio mese
- Top categorie mese
- Grafico trend 6 mesi

### 6.3 Widget avanzati (Advanced)

- Balance Sheet sintetico (Assets/Liabilities/Equity)
- Profit & Loss (Income/Expenses)
- Ricorrenti: previsione "fisso del mese"
- Budget: semaforo per categoria
- Inbox pagamenti attesi (da approvare)

### 6.4 Idee UI

- "Sembra un gioco": mantenere stile leggero/ironico ma leggibile.
- Attenzione al tema scuro e guideline moderne (es. "liquid glass" / effetti trasparenza): soprattutto per icona e palette.

---

## 7) KPI (spiegati + come calcolarli)

> I KPI si calcolano su un periodo (mese, anno) aggregando i conti `income/expense` e derivando il resto.

### 7.1 KPI base

**Income (Totale Entrate)**

- Somma dei movimenti sui conti Income (di norma: Credit su Income).

**Expenses (Totale Spese)**

- Somma dei movimenti sui conti Expense (di norma: Debit su Expense).

**Net Savings (Risparmio netto)**

- `NetSavings = Income - Expenses`

**Savings Rate**

- `SavingsRate = (NetSavings / Income) * 100` (se Income=0 → N/A)

### 7.2 KPI "operativi" (quelli che servono davvero)

**Burn Rate**

- `BurnRate = Expenses / Days`
- (Days = giorni del periodo o giorni trascorsi nel mese corrente)

**Forecast spesa fine mese**

- `ForecastExpenses = BurnRate * DaysInMonth`

**Forecast saldo fine mese (MVP)**

- `ForecastBalance = CurrentBalance - (BurnRate * RemainingDays)`
- Evoluzione: aggiungere entrate attese + ricorrenti

**Ricorrenti vs Variabili**

- `RecurringExpenses = Σ spese da regole ricorrenti (o categorie marcate)`
- `VariableExpenses = Expenses - RecurringExpenses`
- `%Recurring = RecurringExpenses / Expenses * 100`

**Costo mensile normalizzato (annuali/periodiche)**

- `NormalizedMonthly = Amount / CoverageMonths`
- (es. assicurazione 600/12 = 50€ al mese)

**Distribuzione per categoria**

- `CatSpend[c] = Σ spese per categoria`
- `%Cat = CatSpend[c] / Expenses * 100`

### 7.3 KPI "utility trend" (Gas/bollette)

Per supportare "andamento storico del Gas" in modo pulito:

- standardizzare il merchant (Payee) con alias
- facoltativo: salvare "metrica" ($m^3$/kWh) in un campo extra per le bollette

---

## 8) Lingue, preferenze e UX (pratico)

- Lingua: **IT + EN**, default = lingua del sistema (override manuale).
- Preferenze da salvare:
  - modalità Classic/Advanced
  - valuta di default
  - lingua
  - tema (auto/chiaro/scuro)
- Login: per ora **semplice** (anche sviluppo senza login), ma prevedere:
  - email/password
  - email con codice
  - Google / Apple
  - FaceID/Impronta per sblocco app
  - (idea futura) PayPal login + import movimenti PayPal

---

## 9) Architettura (pronta per web + mobile)

### 9.1 Modalità Offline (requisito) + Sync "seamless"

**Problema emerso:** non vogliamo che l'app mobile debba essere connessa a Internet *sempre*.
**Soluzione:** approccio **offline-first** con **coda di sincronizzazione**.

#### Comportamento desiderato (UX)

- L'utente può:
  - aggiungere/modificare transazioni
  - consultare dashboard e lista
  - approvare istanze attese (Inbox) anche **senza rete**.
- Quando torna online:
  - la sincronizzazione parte automaticamente
  - avviene "sotto" in modo invisibile (seamless)
  - l'utente non deve gestire processi manuali, salvo errori/ conflitti.

#### Logica di sincronizzazione (alto livello)

- Ogni modifica locale genera un "evento" da inviare al backend (es. `CreateTransaction`, `UpdateSplit`, `ApproveExpectedInstance`).
- Gli eventi finiscono in una **outbox** locale (coda).
- Quando online:
  1. invio degli eventi in ordine (retry con backoff)
  2. conferma dal server (ack)
  3. rimozione evento dalla coda
  4. pull di eventuali aggiornamenti server (delta) per allineare lo stato

#### Gestione conflitti (semplice ma efficace)

- Ogni record ha:
  - `version` (o `updated_at`)
  - `client_id` (device) e `operation_id` (idempotenza)
- Se due device modificano lo stesso dato:
  - regola MVP: **last-write-wins** + log conflitto
  - evoluzione: schermata "Conflitti" (scelta manuale) per casi importanti

#### Nota implementativa

- Web app può essere PWA con cache/offline di base (lettura e operazioni limitate).
- App mobile: offline completo (DB locale) + sync.

# 10) Bozza struttura database (tabelle + colonne + PK/FK)

```sql
CREATE TABLE users (
  id              UUID PRIMARY KEY,
  email           TEXT UNIQUE NULL,
  password_hash   TEXT NULL,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```sql
CREATE TABLE user_settings (
  user_id         UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  preferred_mode  TEXT NOT NULL DEFAULT 'classic' CHECK (preferred_mode IN
('classic','advanced')),
  preferred_lang  TEXT NOT NULL DEFAULT 'system',
  preferred_theme TEXT NOT NULL DEFAULT 'system',
  default_currency TEXT NOT NULL DEFAULT 'EUR',
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE auth_identities (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  provider        TEXT NOT NULL,
  provider_uid    TEXT NOT NULL,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(provider, provider_uid)
);

CREATE TABLE ledger_accounts (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  parent_id       UUID NULL REFERENCES ledger_accounts(id) ON DELETE SET
NULL,
  name            TEXT NOT NULL,
  kind            TEXT NOT NULL CHECK (kind IN
('asset','liability','equity','income','expense')),
  currency        TEXT NULL,
  is_archived     BOOLEAN NOT NULL DEFAULT FALSE,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (user_id, parent_id, name)
);

CREATE TABLE payees (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  canonical_name  TEXT NOT NULL,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(user_id, canonical_name)
);

CREATE TABLE payee_aliases (
  id              UUID PRIMARY KEY,
  payee_id        UUID NOT NULL REFERENCES payees(id) ON DELETE CASCADE,
  alias           TEXT NOT NULL,
  UNIQUE(payee_id, alias)
);

CREATE TABLE transactions (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  txn_date        DATE NOT NULL,
  description     TEXT NULL,
```

```sql
  payee_id        UUID NULL REFERENCES payees(id) ON DELETE SET NULL,
  source          TEXT NOT NULL DEFAULT 'manual'
                  CHECK (source IN
('manual','import','recurring','adjustment','opening')),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_tx_user_date ON transactions(user_id, txn_date);

CREATE TABLE splits (
  id              UUID PRIMARY KEY,
  transaction_id  UUID NOT NULL REFERENCES transactions(id) ON DELETE
CASCADE,
  account_id      UUID NOT NULL REFERENCES ledger_accounts(id) ON DELETE
RESTRICT,
  direction       TEXT NOT NULL CHECK (direction IN ('debit','credit')),
  amount          NUMERIC(18,2) NOT NULL CHECK (amount >= 0),
  memo            TEXT NULL
);

CREATE INDEX idx_splits_txn ON splits(transaction_id);
CREATE INDEX idx_splits_account ON splits(account_id);

CREATE TABLE tags (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name            TEXT NOT NULL,
  UNIQUE (user_id, name)
);

CREATE TABLE transaction_tags (
  transaction_id  UUID NOT NULL REFERENCES transactions(id) ON DELETE
CASCADE,
  tag_id          UUID NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
  PRIMARY KEY (transaction_id, tag_id)
);

CREATE TABLE attachments (
  id              UUID PRIMARY KEY,
  transaction_id  UUID NOT NULL REFERENCES transactions(id) ON DELETE
CASCADE,
  storage_ref     TEXT NOT NULL,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE transaction_metrics (
  id              UUID PRIMARY KEY,
  transaction_id  UUID NOT NULL REFERENCES transactions(id) ON DELETE
CASCADE,
  metric_type     TEXT NOT NULL,
  metric_value    NUMERIC(18,4) NOT NULL,
  unit            TEXT NOT NULL,
  UNIQUE(transaction_id, metric_type)
);
```

```sql
CREATE TABLE scheduled_rules (
  id                  UUID PRIMARY KEY,
  user_id             UUID NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
  name                TEXT NOT NULL,
  pay_account_id      UUID NOT NULL REFERENCES ledger_accounts(id),
  counter_account_id  UUID NOT NULL REFERENCES ledger_accounts(id),
  frequency           TEXT NOT NULL CHECK (frequency IN
('weekly','monthly','yearly','custom')),
  interval_n          INT  NOT NULL DEFAULT 1,
  day_of_month        INT  NULL CHECK (day_of_month BETWEEN 1 AND 28),
  start_date          DATE NOT NULL,
  end_date            DATE NULL,
  amount_strategy     TEXT NOT NULL CHECK (amount_strategy IN
('fixed','last_month','avg_n','estimate_reconcile','schedule_table')),
  fixed_amount        NUMERIC(18,2) NULL,
  avg_n               INT NULL,
  estimate_amount     NUMERIC(18,2) NULL,
  tolerance_amount    NUMERIC(18,2) NOT NULL DEFAULT 2.00,
  is_active           BOOLEAN NOT NULL DEFAULT TRUE,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE expected_instances (
  id              UUID PRIMARY KEY,
  rule_id         UUID NOT NULL REFERENCES scheduled_rules(id) ON DELETE
CASCADE,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  due_date        DATE NOT NULL,
  proposed_amount NUMERIC(18,2) NOT NULL,
  status          TEXT NOT NULL DEFAULT 'pending'
                  CHECK (status IN
('pending','approved','matched','skipped','expired')),
  approved_at     TIMESTAMPTZ NULL,
  matched_txn_id  UUID NULL REFERENCES transactions(id) ON DELETE SET
NULL,
  note            TEXT NULL
);

CREATE TABLE loan_schedules (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  rule_id         UUID NULL REFERENCES scheduled_rules(id) ON DELETE SET
NULL,
  name            TEXT NOT NULL,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE loan_schedule_items (
  id              UUID PRIMARY KEY,
  schedule_id     UUID NOT NULL REFERENCES loan_schedules(id) ON DELETE
CASCADE,
  due_date        DATE NOT NULL,
```

```
  amount          NUMERIC(18,2) NOT NULL,
  principal       NUMERIC(18,2) NULL,
  interest        NUMERIC(18,2) NULL
);

CREATE TABLE budgets (
  id                  UUID PRIMARY KEY,
  user_id             UUID NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
  expense_account_id  UUID NOT NULL REFERENCES ledger_accounts(id),
  period              TEXT NOT NULL,
  amount              NUMERIC(18,2) NOT NULL CHECK (amount >= 0),
  UNIQUE (user_id, expense_account_id, period)
);

CREATE TABLE dashboard_widgets (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  widget_type     TEXT NOT NULL,
  position        INT  NOT NULL,
  config_json     JSONB NOT NULL DEFAULT '{}'::jsonb,
  UNIQUE (user_id, position)
);

CREATE TABLE import_jobs (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  source_type     TEXT NOT NULL CHECK (source_type IN
('gnucash','money_manager','csv','xlsx')),
  filename        TEXT NOT NULL,
  status          TEXT NOT NULL DEFAULT 'uploaded'
                  CHECK (status IN
('uploaded','parsed','mapped','previewed','committed','failed')),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  error_message   TEXT NULL
);

CREATE TABLE import_rows (
  id              UUID PRIMARY KEY,
  import_job_id   UUID NOT NULL REFERENCES import_jobs(id) ON DELETE
CASCADE,
  raw_json        JSONB NOT NULL,
  row_hash        TEXT NULL,
  status          TEXT NOT NULL DEFAULT 'new'
                  CHECK (status IN ('new','duplicate','mapped','error')),
  mapped_txn_id   UUID NULL REFERENCES transactions(id) ON DELETE SET
NULL,
  error_message   TEXT NULL
);

-- DEVICES (per sync multi-device + diagnostica)
CREATE TABLE devices (
  id              UUID PRIMARY KEY,
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
```

```sql
  device_name     TEXT NULL,
  platform        TEXT NOT NULL, -- 'ios'|'android'|'web'
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_seen_at    TIMESTAMPTZ NULL
);

-- OPERATION LOG (idempotenza: stesso evento non deve applicarsi 2 volte)
CREATE TABLE operation_log (
  id              UUID PRIMARY KEY, -- operation_id
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  device_id       UUID NULL REFERENCES devices(id) ON DELETE SET NULL,
  op_type         TEXT NOT NULL,
  entity_type     TEXT NOT NULL,
  entity_id       UUID NULL,
  applied_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_oplog_user_applied ON operation_log(user_id, applied_at);

-- SERVER SYNC CURSOR (per sync "delta" dal server al device)
CREATE TABLE sync_cursors (
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  device_id       UUID NOT NULL REFERENCES devices(id) ON DELETE CASCADE,
  last_sync_at    TIMESTAMPTZ NULL,
  PRIMARY KEY (user_id, device_id)
);
```

---

## 11) Piano (da cui fare backlog)

1. Core accounting + template conti + opening balances + preferenze (Classic/Advanced)
2. Import GnuCash/Money Manager con staging/preview
3. Ricorrenti variabili: scheduled + inbox + strategie importo
4. Dashboard componibile + KPI + trend “Gas” (payee/alias + metriche)
5. Export/report (seconda fase)